

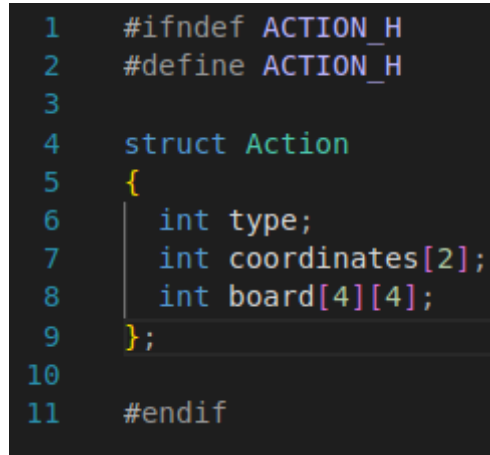
Trabalho Prático 1 - Campo Minado

Lucas Pereira Carvalho - 2020054676

Para a entrega do trabalho prático 1 da disciplina de Redes de Computadores, foi requisitado o desenvolvimento de um jogo conhecido como campo minado, aplicado ao contexto da disciplina, de forma que houvesse uma comunicação do jogador (cliente) com um servidor - este por sua vez manteria um estado do jogo e o atualizaria após cada jogada do cliente.

A comunicação com o servidor faz-se necessária, uma vez que é nele que existe um board de gabarito, ou seja, um tabuleiro indicando onde estão as bombas. A cada jogada do cliente, uma mensagem é trafegada pela rede, o servidor a recebe e aplica a lógica do jogo campo minado, para verificar se a jogada é válida, e indicar caso ocorra uma vitória (quando todas células exceto bombas são reveladas) ou caso ocorra uma derrota (quando jogador tenta revelar uma casa com bomba).

A mensagem trafegada pela rede foi uma struct (como pode-se ver na imagem abaixo),

A screenshot of a code editor showing the definition of an Action struct in C. The code is as follows:

```
1  #ifndef ACTION_H
2  #define ACTION_H
3
4  struct Action
5  {
6      int type;
7      int coordinates[2];
8      int board[4][4];
9  };
10
11 #endif
```

uma vez que todo o sistema foi desenvolvido na linguagem C. O campo type indica que tipo de ação está sendo trafegada na mensagem, as coordenadas indicam em qual casa a ação ocorreu, e o board é usado quando o servidor envia uma mensagem para o cliente, após atualizar o estado do tabuleiro do jogo. A partir do estabelecimento de um padrão de mensagem trafegada, torna-se mais simples o desenvolvimento da comunicação entre o par cliente-servidor.

O jogo desenvolvido segue as mesmas regras do clássico jogo. Neste caso, trata-se de um tabuleiro 4x4, com 3 bombas, ou seja, revelando as demais 13 casas o jogo é vencido. Caso revele uma bomba - derrota. É possível marcar casas que o usuário deduza que seja uma bomba, assim como remover esta marcação. O jogo pode ser reiniciado a qualquer momento, inclusive após uma possível derrota.

Após o devido entendimento das regras do jogo, e como as duas entidades irão se comunicar, foi iniciado o desenvolvimento da comunicação em si do programa servidor e do programa cliente. Utilizando da biblioteca POSIX, é possível implementar e configurar a

comunicação por meio de sockets na linguagem C. Esta etapa foi realizada com suporte da série de vídeos publicada pelo professor Ítalo Cunha. Baseando-se no código apresentado pelo professor em seus vídeos, foi possível estabelecer uma comunicação entre dois programas.

No servidor, primeiramente é instanciado uma struct do tipo *sockaddr_storage* para armazenar o endereço do mesmo. Um ponto a se destacar, foi a preocupação em implementar os dois protocolos de endereçamento IP, o sistema entregue funciona utilizando tanto IPv4, quanto IPv6. De acordo com o argumento de linha de comando ao executar o servidor, o protocolo de endereçamento é devidamente configurado.

Após isso, faz-se um bind do socket do servidor, ao socket que irá se comunicar com o cliente. Com o listen, é possível detectar as mensagens enviadas pelo cliente ao servidor, e com o comando accept, como o próprio nome indica, é permitida a comunicação entre ambas as partes.

O comando recv fica encarregado de receber a mensagem do cliente, e armazenar numa struct local do servidor. Após isso, a sequência de passos, e a lógica aplicada em cada uma é a seguinte:

1. O cliente a partir do código da jogada (0 para start, 1 para reveal, 2 para flag de uma casa, 4 para remoção de flag, 5 para reiniciar o jogo e 6 para sair do jogo), valida os outros possíveis parâmetros, e demais detalhes.
- 2.1. No caso de reveal, verifica se a casa no gabarito é uma bomba. Caso seja, fim de jogo - note-se que o jogador pode reiniciar o jogo e tentar novamente. Caso contrário, diminui-se uma das 13 casas iniciais a serem reveladas.
- 2.2. No caso de flag e remoção de flag, o board de estado do jogo é atualizado, adicionando ou removendo a flag na casa passada.
- 2.3. Caso seja um reset, o board de estado do jogo é reinicializado como um novo jogo, assim como as 13 casas a serem reveladas.
- 2.4. Caso seja um exit, o cliente se desconecta do servidor. O servidor por sua vez, volta para etapa de escutar por uma conexão de algum cliente.

Após a computação da jogada no lado do servidor, e a devida atualização do estado do jogo, o servidor envia esta atualização para o cliente com o comando *send*, e o socket configurado para o servidor escuta e imprime no console o tabuleiro recém atualizado, ou em caso de derrota/ vitória/reset de jogo, imprime a devida mensagem de aviso.

Existem validações de comandos no lado do cliente antes de a mensagem ser enviada para o servidor. O cliente não permite comandos que não estejam mapeados no jogo. Não permite ação em uma célula fora dos limites do tabuleiro 4x4. Também não permite revelar uma célula já revelada, assim como colocar flag em uma célula que já possua flag, ou então que já tenha sido revelada. Em demais casos a mensagem chega a ser enviada ao servidor, mas caso seja alguma jogada que quebre as regras do jogo, o servidor simplesmente ignora a ação e envia um board exatamente igual ao da última jogada válida. Vale ressaltar que existe um loop para leitura dos comandos do cliente, dessa forma, caso o cliente digite um comando inválido, o erro é apresentado, porém ele ainda consegue continuar a jogar.

Foi implementado um arquivo Makefile na raiz do projeto, dessa forma, para executar o jogo basta primeiramente digitar o comando *make*. Após isso, os arquivos serão compilados e o executável *server* e *client* estarão disponíveis.

A partir do desenvolvimento deste projeto, foi possível uma grande familiarização do desenvolvimento com a biblioteca POSIX, e o melhor entendimento de como funciona a configuração e utilização de sockets para comunicação de dois programas por meio de uma rede. Além disso, foi possível relembrar práticas de desenvolvimento e de estruturas de dados, para o devido desenvolvimento do jogo campo minado.

Referências

Cunha, Ítalo. Playlist de vídeos Introdução a Programação em Redes. Disponível em: <https://www.youtube.com/watch?v=tJ3qNtv0HVs&t=2s>

Bonesteel, Cameron. Repositório público - C minesweeper game. Disponível em: <https://github.com/cbonesteel/c-minesweeper>